

```

1 import pandas as pd
2
3 # Ссылка на наш выбранный датасет
4 url = "https://raw.githubusercontent.com/leonism/sample-superstore/master/data/superstore.csv"
5
6 # Загружаем данные в DataFrame
7 # Примечание: encoding='latin-1' используется здесь, так как оригинальный датасет
8 # Superstore часто содержит специфические символы в названиях, которые вызывают
9 # ошибку при стандартной кодировке utf-8.
10 df = pd.read_csv(url, encoding='latin-1')
11
12 # Смотрим на первые 5 строк, чтобы убедиться, что всё загрузилось корректно
13 df.head()

```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID
0	1	CA-2017-152156	11/8/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420.0	South	FUR-B100017
1	2	CA-2017-152156	11/8/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420.0	South	FUR-C100004
2	3	CA-2017-138688	6/12/2017	6/16/2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036.0	West	OFF-I100002
3	4	US-2016-108966	10/11/2016	10/18/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311.0	South	FUR-F100005
4	5	US-2016-108966	10/11/2016	10/18/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311.0	South	OFF-F100007

5 rows × 21 columns

```
1 df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10800 entries, 0 to 10799
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 10800 non-null object
1   Order ID               10800 non-null object
2   Order Date             9994 non-null  object
3   Ship Date              9994 non-null  object
4   Ship Mode              9994 non-null  object
5   Customer ID            9994 non-null  object
6   Customer Name          9994 non-null  object
7   Segment                9994 non-null  object
8   Country                9994 non-null  object
9   City                   9994 non-null  object
10  State                  9994 non-null  object
11  Postal Code            9983 non-null  float64
12  Region                 9994 non-null  object
13  Product ID             9994 non-null  object
14  Category               9994 non-null  object
15  Sub-Category           9994 non-null  object
16  Product Name           9994 non-null  object
17  Sales                  9994 non-null  float64
18  Quantity               9994 non-null  float64
19  Discount                9994 non-null  float64
20  Profit                 9994 non-null  float64
dtypes: float64(5), object(16)
memory usage: 1.7+ MB

```

```
1 df.describe()
```

	Postal Code	Sales	Quantity	Discount	Profit
count	9983.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	55245.233297	229.858001	3.789574	0.156203	28.656896
std	32038.715955	623.245101	2.225110	0.206452	234.260108
min	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	57103.000000	54.490000	3.000000	0.200000	8.666500
75%	90008.000000	209.940000	5.000000	0.200000	29.364000
max	99301.000000	22638.480000	14.000000	0.800000	8399.976000

«Определить ключевые факторы, влияющие на общую выручку и чистую прибыль розничной сети, выявить наиболее и наименее прибыльные категории товаров, а также проанализировать сезонные колебания продаж за представленный временной период».

```

1 # 1. Смотрим на 3 случайные строки, чтобы оценить реальные "сырые" данные
2 print("Случайные строки датасета:")
3 display(df.sample(3))
4
5 # 2. Находим столбцы, которые могут быть географическими, и считаем в них уникальные значения
6 # Это поможет нам понять масштаб для будущей карты
7 geo_candidates = [col for col in df.columns if col.lower() in ['country', 'state', 'city', 'region', 'postal code']]
8
9 if geo_candidates:
10     print("\nКоличество уникальных значений в географических столбцах:")
11     display(df[geo_candidates].nunique())
12 else:
13     print("\nСтандартные географические столбцы не найдены. Проверим уникальные значения во всех столбцах:")
14     display(df.nunique())

```

Случайные строки датасета:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	P
3827	3828	CA-2018-141733	5/7/2018	5/11/2018	Standard Class	RW-19540	Rick Wilson	Corporate	United States	Detroit	...	48234.0	Central	FL 10
598	599	CA-2017-120180	7/14/2017	7/16/2017	First Class	TP-21130	Theone Pippenger	Consumer	United States	Philadelphia	...	19134.0	East	O 10
3618	3619	CA-2018-141929	9/4/2018	9/8/2018	Standard Class	RA-19285	Ralph Arnett	Consumer	United States	Los Angeles	...	90004.0	West	TI 10

3 rows × 21 columns

Количество уникальных значений в географических столбцах:

	0
Country	1
City	531
State	49
Postal Code	630
Region	4

dtype: int64

```

1 # 1. Проверяем количество пропущенных значений в каждом столбце
2 print("Пропущенные значения по столбцам:")
3 print(df.isnull().sum())
4
5 # 2. Проверяем наличие полных дубликатов строк
6 print(f"\nКоличество полных дубликатов строк: {df.duplicated().sum()}")

```

```
7
8 # 3. Базовая проверка на аномалии в ключевых числовых столбцах
9 # Смотрим минимальные значения, чтобы понять, есть ли отрицательные продажи или странные количества
10 print("\nМинимальные значения в ключевых столбцах:")
```

Пропущенные значения по столбцам:

```
Row ID      0
Order ID    0
Order Date  806
Ship Date   806
Ship Mode   806
Customer ID 806
Customer Name 806
Segment     806
Country     806
City        806
State       806
Postal Code 817
Region      806
Product ID  806
Category    806
Sub-Category 806
Product Name 806
Sales       806
Quantity    806
Discount    806
Profit      806
dtype: int64
```

Количество полных дубликатов строк: 504

Минимальные значения в ключевых столбцах:

```
Quantity    1.000
Sales       0.444
Profit     -6599.978
dtype: float64
```

```
1 # 1. Удаляем полные дубликаты строк (если они были найдены) и сбрасываем индекс
2 df = df.drop_duplicates().reset_index(drop=True)
3
4 # 2. Преобразуем столбец 'Order Date' в формат даты (datetime)
5 # Формат дат в этом датасете обычно 'MM/DD/YYYY', pandas обычно распознает это автоматически,
6 # но мы явно указываем формат для надежности, если потребуется.
7 df['Order Date'] = pd.to_datetime(df['Order Date'])
8
9 # 3. Создаем новые полезные столбцы на основе даты для удобства анализа сезонности
10 df['Year'] = df['Order Date'].dt.year
11 df['Month'] = df['Order Date'].dt.month
12 df['YearMonth'] = df['Order Date'].dt.to_period('M')
13
14 # 4. Проверяем результат преобразования
15 print("Типы данных ключевых столбцов после очистки:")
16 print(df[['Order Date', 'Year', 'Month', 'Sales', 'Profit']].dtypes)
17
18 print("\nПервые 5 строк с новыми временными столбцами:")
19 display(df[['Order Date', 'Year', 'Month', 'Sales', 'Profit']].head())
20 df.head()
```

Типы данных ключевых столбцов после очистки:

```
Order Date    datetime64[ns]
Year          float64
Month         float64
Sales         float64
Profit        float64
dtype: object
```

Первые 5 строк с новыми временными столбцами:

	Order Date	Year	Month	Sales	Profit
0	2017-11-08	2017.0	11.0	261.9600	41.9136
1	2017-11-08	2017.0	11.0	731.9400	219.5820
2	2017-06-12	2017.0	6.0	14.6200	6.8714
3	2016-10-11	2016.0	10.0	957.5775	-383.0310
4	2016-10-11	2016.0	10.0	22.3680	2.5164

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Category	Sub-Category	Product
0	1	CA-2017-152156	2017-11-08	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	Furniture	Bookcases	Sonoma Collapsible Bookshelf
1	2	CA-2017-152156	2017-11-08	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	Furniture	Chairs	Honoring Chair Upholstered Chair
2	3	CA-2017-138688	2017-06-12	6/16/2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	Office Supplies	Labels	Address Labels Adhesive Label Typevarious
3	4	US-2016-108966	2016-10-11	10/18/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	Furniture	Tables	Brass Coffee Table Series Rectangular
4	5	US-2016-108966	2016-10-11	10/18/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	Office Supplies	Storage	Eldorado Storage Cabinet

```
1 # 1. Общие показатели бизнеса
2 total_sales = df['Sales'].sum()
3 total_profit = df['Profit'].sum()
4 # Защита от деления на ноль, если вдруг выручка равна 0
5 profit_margin = (total_profit / total_sales * 100) if total_sales != 0 else 0
6
7 print("=== ОБЩИЕ ПОКАЗАТЕЛИ ===")
8 print(f"Общая выручка (Sales): {total_sales:,.2f}")
9 print(f"Общая прибыль (Profit): {total_profit:,.2f}")
10 print(f"Рентабельность (Profit Margin): {profit_margin:,.2f}%\n")
11
12 # 2. Анализ по категориям товаров (выручка, прибыль и средняя скидка)
13 category_stats = df.groupby('Category').agg(
14     Total_Sales=('Sales', 'sum'),
15     Total_Profit=('Profit', 'sum'),
16     Avg_Discount=('Discount', 'mean')
17 ).round(2).sort_values(by='Total_Profit', ascending=False)
18
19 print("=== ПОКАЗАТЕЛИ ПО КАТЕГОРИЯМ ===")
20 display(category_stats)
21
22 # 3. Сезонность (агрегация продаж по месяцам)
23 monthly_sales = df.groupby('Month')['Sales'].sum().reset_index()
24 # Сортируем по месяцу для хронологического порядка
25 monthly_sales = monthly_sales.sort_values('Month')
26
27 print("\n=== ПРОДАЖИ ПО МЕСЯЦАМ (СЕЗОННОСТЬ) ===")
28 display(monthly_sales)
29
30 # 4. География (подготовка данных для карты: агрегация по штатам)
31 # В классическом датасете Superstore столбец штата называется 'State'
32 if 'State' in df.columns:
33     state_sales = df.groupby('State')['Sales'].sum().sort_values(ascending=False).reset_index()
34     print("\n=== ТОП-10 ШТАТОВ ПО ВЫРУЧКЕ (для будущей карты) ===")
35     display(state_sales.head(10))
36 else:
```

```
37 print("\nВнимание: столбец 'State' не найден. Доступные столбцы:", df.columns.tolist())
```

=== ОБЩИЕ ПОКАЗАТЕЛИ ===

Общая выручка (Sales): 2,297,200.86
Общая прибыль (Profit): 286,397.02
Рентабельность (Profit Margin): 12.47%

=== ПОКАЗАТЕЛИ ПО КАТЕГОРИЯМ ===

	Total_Sales	Total_Profit	Avg_Discount	
Category				
Technology	836154.03	145454.95	0.13	
Office Supplies	719047.03	122490.80	0.16	
Furniture	741999.80	18451.27	0.17	

=== ПРОДАЖИ ПО МЕСЯЦАМ (СЕЗОННОСТЬ) ===

	Month	Sales
0	1.0	94924.8356
1	2.0	59751.2514
2	3.0	205005.4888
3	4.0	137762.1286
4	5.0	155028.8117
5	6.0	152718.6793
6	7.0	147238.0970
7	8.0	159044.0630
8	9.0	307649.9457
9	10.0	200322.9847
10	11.0	352461.0710
11	12.0	325293.5035

=== ТОП-10 ШТАТОВ ПО ВЫРУЧКЕ (для будущей карты) ===

	State	Sales
0	California	457687.6315
1	New York	310876.2710
2	Texas	170188.0458
3	Washington	138641.2700
4	Pennsylvania	116511.9140
5	Florida	89473.7080
6	Illinois	80166.1010
7	Ohio	78258.1360
8	Michigan	76269.6140
9	Virginia	70636.7200

Далее:

[New interactive sheet](#)

[New interactive sheet](#)

```
1 import matplotlib.pyplot as plt
2
3 # Настраиваем размер и стиль графиков для лучшей читаемости
4 plt.style.use('default')
5
6 # 1. СЕЗОННОСТЬ: Линейный график продаж по месяцам
7 plt.figure(figsize=(10, 5))
8 plt.plot(monthly_sales['Month'], monthly_sales['Sales'], marker='o', color='blue', linewidth=2)
9 plt.title('Динамика выручки по месяцам (Сезонность)', fontsize=14)
10 plt.xlabel('Месяц', fontsize=12)
11 plt.ylabel('Выручка (Sales)', fontsize=12)
12 plt.xticks(range(1, 13)) # Показываем все 12 месяцев на оси X
13 plt.grid(True, linestyle='--', alpha=0.7)
14 plt.show()
```

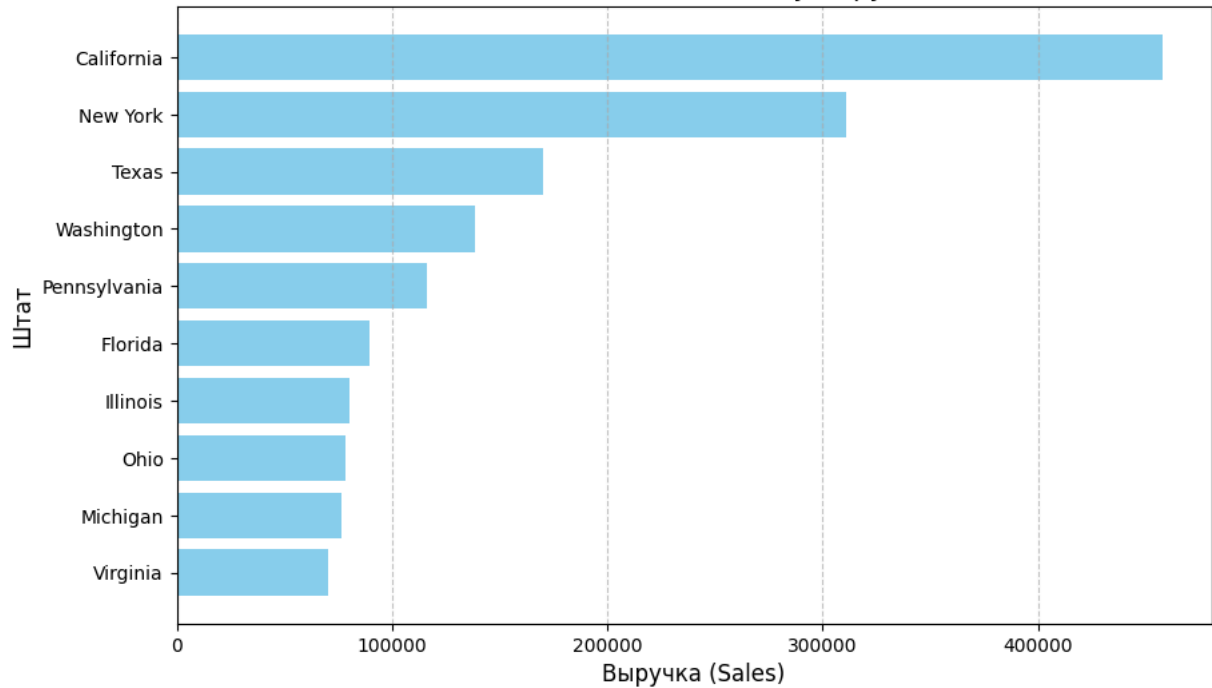


```
1
2 # 2. КАТЕГОРИИ: Столбчатая диаграмма прибыли по категориям
3 # Подсветим зеленым прибыльные категории, красным - убыточные
4 plt.figure(figsize=(8, 5))
5 colors = ['green' if x > 0 else 'red' for x in category_stats['Total_Profit']]
6 plt.bar(category_stats.index, category_stats['Total_Profit'], color=colors)
7 plt.title('Чистая прибыль по категориям товаров', fontsize=14)
8 plt.xlabel('Категория', fontsize=12)
9 plt.ylabel('Прибыль (Profit)', fontsize=12)
10 plt.xticks(rotation=0)
11 plt.grid(axis='y', linestyle='--', alpha=0.7)
12 plt.show()
```



```
1 # 3. ГЕОГРАФИЯ (Вариант А): Горизонтальная диаграмма Топ-10 штатов в matplotlib
2 plt.figure(figsize=(10, 6))
3 top_states = state_sales.head(10)
4 plt.barh(top_states['State'], top_states['Sales'], color='skyblue')
5 plt.title('Топ-10 штатов по объему выручки', fontsize=14)
6 plt.xlabel('Выручка (Sales)', fontsize=12)
7 plt.ylabel('Штат', fontsize=12)
8 plt.gca().invert_yaxis() # Инвертируем, чтобы самый большой столбец был сверху
9 plt.grid(axis='x', linestyle='--', alpha=0.7)
10 plt.show()
```

Топ-10 штатов по объему выручки



```

1 import plotly.express as px
2 # 1. Исправление для географической карты: создаем словарь кодов штатов
3 state_codes = {
4     'California': 'CA', 'New York': 'NY', 'Texas': 'TX', 'Florida': 'FL',
5     'Illinois': 'IL', 'Pennsylvania': 'PA', 'Ohio': 'OH', 'Georgia': 'GA',
6     'North Carolina': 'NC', 'Michigan': 'MI', 'New Jersey': 'NJ', 'Virginia': 'VA',
7     'Washington': 'WA', 'Massachusetts': 'MA', 'Arizona': 'AZ', 'Indiana': 'IN',
8     'Tennessee': 'TN', 'Missouri': 'MO', 'Maryland': 'MD', 'Wisconsin': 'WI',
9     'Minnesota': 'MN', 'Colorado': 'CO', 'Alabama': 'AL', 'South Carolina': 'SC',
10    'Louisiana': 'LA', 'Kentucky': 'KY', 'Oregon': 'OR', 'Oklahoma': 'OK',
11    'Connecticut': 'CT', 'Utah': 'UT', 'Nevada': 'NV', 'Arkansas': 'AR',
12    'Mississippi': 'MS', 'Kansas': 'KS', 'New Mexico': 'NM', 'Nebraska': 'NE',
13    'West Virginia': 'WV', 'Idaho': 'ID', 'Hawaii': 'HI', 'New Hampshire': 'NH',
14    'Maine': 'ME', 'Montana': 'MT', 'Rhode Island': 'RI', 'Delaware': 'DE',
15    'South Dakota': 'SD', 'North Dakota': 'ND', 'Alaska': 'AK', 'Vermont': 'VT',
16    'Wyoming': 'WY', 'District of Columbia': 'DC'
17 }
18
19 # Создаем копию таблицы для карты и добавляем столбец с кодами
20 map_data = state_sales.copy()
21 map_data['State_Code'] = map_data['State'].map(state_codes)
22
23 # 2. Строим исправленную географическую карту
24 fig_map = px.choropleth(
25     map_data,
26     locations='State_Code',
27     locationmode='USA-states',
28     color='Sales',
29     scope="usa",
30     title='География продаж по штатам США',
31     color_continuous_scale='Blues',
32     hover_name='State' # При наведении мыши будет показываться полное название штата
33 )
34 fig_map.show()

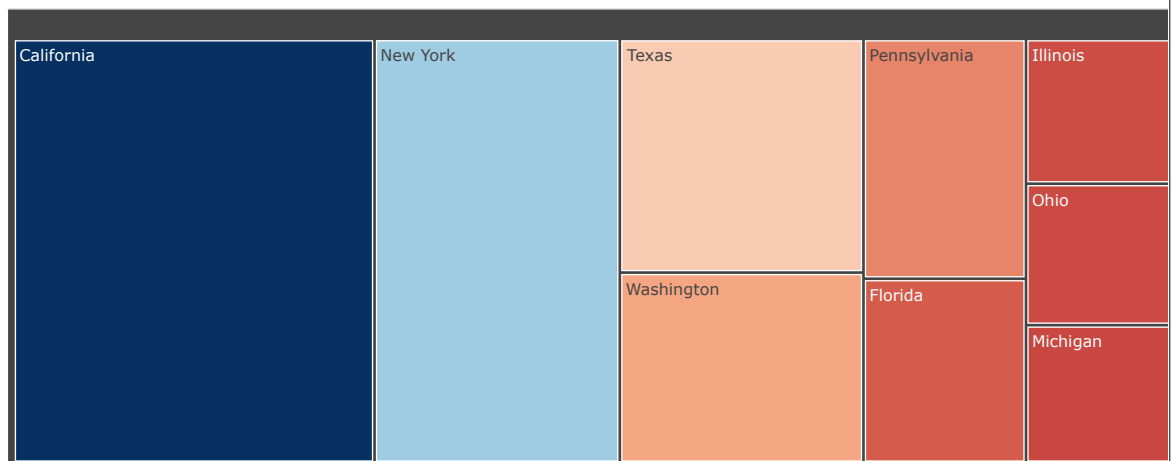
```

География продаж по штатам США



```
1 # 3. БОНУС: Treemap (Древовидная диаграмма)
2 # Она не зависит от географических границ, никогда не "ломается" и наглядно показывает долю каждого штата
3 fig_tree = px.treemap(
4     state_sales,
5     path=['State'],
6     values='Sales',
7     color='Sales',
8     color_continuous_scale='RdBu', # Красно-синяя шкала: красный = мало, синий = много
9     title='Доля штатов в общей выручке (Treemap)'
10 )
11 fig_tree.show()
```

Доля штатов в общей выручке (Treemap)



```
1 # Проверим точные временные рамки датасета
2 min_date = df['Order Date'].min().strftime('%Y-%m-%d')
3 max_date = df['Order Date'].max().strftime('%Y-%m-%d')
4
5 print(f"=== ОГРАНИЧЕНИЯ АНАЛИЗА ===")
6 print(f"1. Временные рамки: Данные охватывают только период с {min_date} по {max_date}.")
7 print("   (Рыночные условия, инфляция или поведение клиентов могли измениться после этой даты).")
8 print("2. Отсутствие внешних факторов: В датасете нет информации о маркетинговых бюджетах, действиях конкурентов или м")
9 print("3. Географическая специфика: Данные, скорее всего, относятся к одному конкретному рынку (например, США), и вывод")
10 print("4. Уровень детализации: Мы видим агрегированные заказы, но не знаем глубинных причин возврата товаров или точног")
```

=== ОГРАНИЧЕНИЯ АНАЛИЗА ===

1. Временные рамки: Данные охватывают только период с 2015-01-03 по 2018-12-30.
(Рыночные условия, инфляция или поведение клиентов могли измениться после этой даты).
2. Отсутствие внешних факторов: В датасете нет информации о маркетинговых бюджетах, действиях конкурентов или макроэкономике.
3. Географическая специфика: Данные, скорее всего, относятся к одному конкретному рынку (например, США), и выводы могут не применяться к другим.
4. Уровень детализации: Мы видим агрегированные заказы, но не знаем глубинных причин возврата товаров или точного пути клиента.

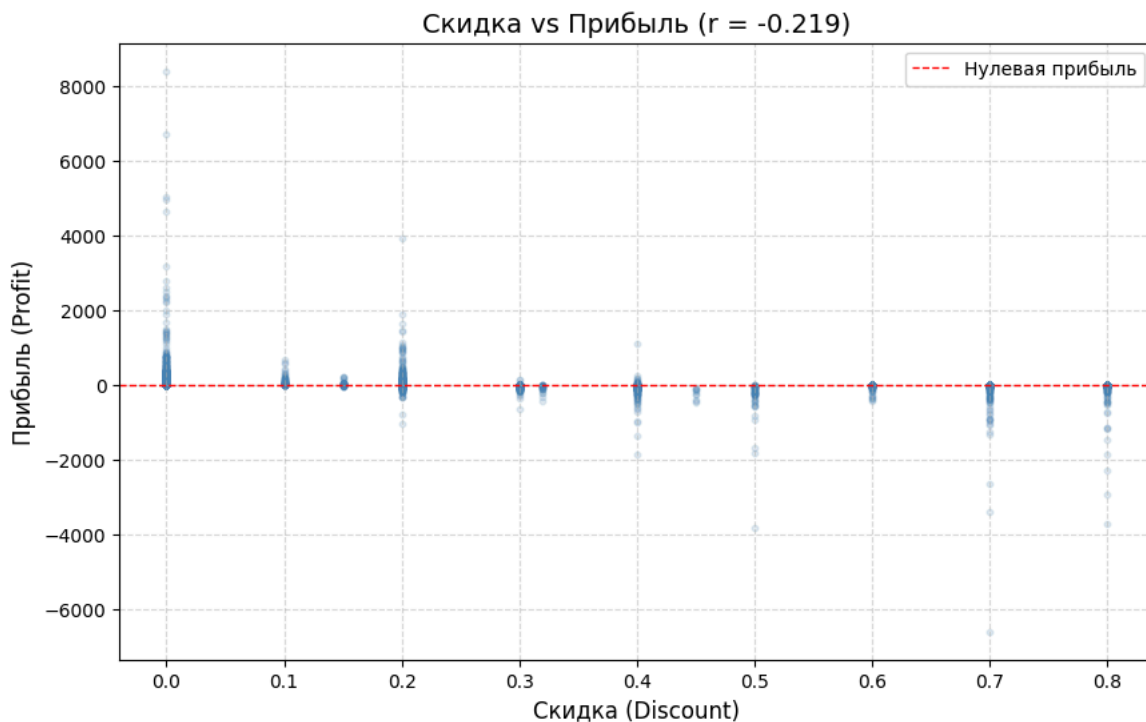
```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # 1.1. Рассчитываем коэффициент корреляции Пирсона между Discount и Profit
5 correlation = df['Discount'].corr(df['Profit'])
6 print(f"=== КОРРЕЛЯЦИЯ СКИДКИ И ПРИБЫЛИ ===")
7 print(f"Коэффициент корреляции (Pearson): {correlation:.3f}")
8 print(f"Интерпретация: {'отрицательная' if correlation < 0 else 'положительная'} связь.")
9 print(f"Чем ближе к -1, тем сильнее скидки снижают прибыль.\n")
10
11 # 1.2. Scatter plot: каждая точка = один заказ
12 plt.figure(figsize=(10, 6))
13 plt.scatter(df['Discount'], df['Profit'], alpha=0.15, s=10, color='steelblue')
14 plt.axhline(y=0, color='red', linestyle='--', linewidth=1, label='Нулевая прибыль')
15 plt.title(f'Скидка vs Прибыль (r = {correlation:.3f})', fontsize=14)
16 plt.xlabel('Скидка (Discount)', fontsize=12)
17 plt.ylabel('Прибыль (Profit)', fontsize=12)
18 plt.legend()
19 plt.grid(True, linestyle='--', alpha=0.5)
20 plt.show()
```

=== КОРРЕЛЯЦИЯ СКИДКИ И ПРИБЫЛИ ===

Коэффициент корреляции (Pearson): -0.219

Интерпретация: отрицательная связь.

Чем ближе к -1, тем сильнее скидки снижают прибыль.

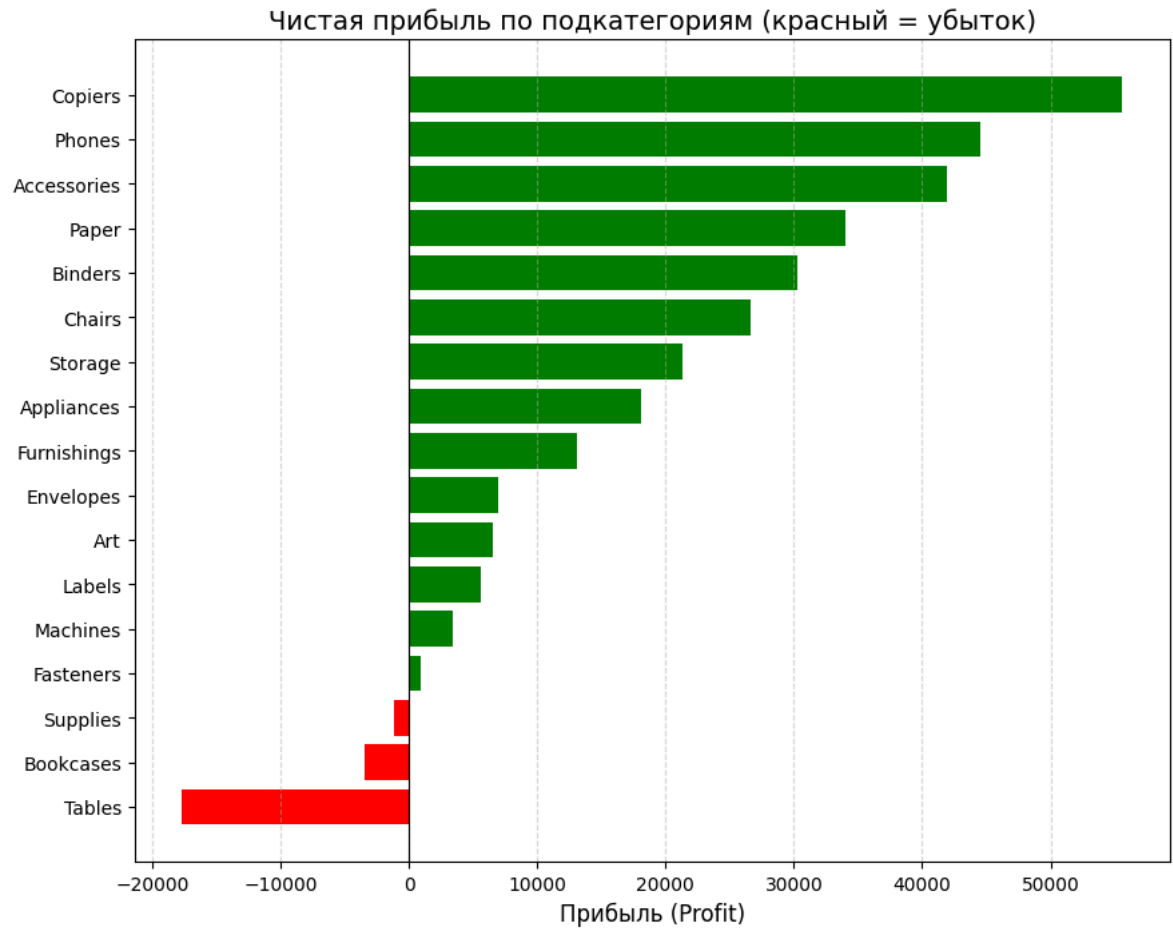


```
1 # 2.1. Сводная таблица по подкатегориям
2 subcategory_stats = df.groupby(['Category', 'Sub-Category']).agg(
3     Total_Sales=('Sales', 'sum'),
4     Total_Profit=('Profit', 'sum'),
5     Avg_Discount=('Discount', 'mean'),
6     Order_Count=('Order ID', 'count')
7 ).round(2).sort_values(by='Total_Profit', ascending=True)
8
9 print("=== ПРИБЫЛЬ ПО ПОДКАТЕГОРИЯМ (от худших к лучшим) ===")
10 display(subcategory_stats)
11
12 # 2.2. Горизонтальная диаграмма: все подкатегории по прибыли
13 plt.figure(figsize=(10, 8))
14 colors = ['green' if x > 0 else 'red' for x in subcategory_stats['Total_Profit']]
15 plt.barh(subcategory_stats.index.get_level_values('Sub-Category'),
16          subcategory_stats['Total_Profit'], color=colors)
17 plt.title('Чистая прибыль по подкатегориям (красный = убыток)', fontsize=14)
18 plt.xlabel('Прибыль (Profit)', fontsize=12)
```

```
19 plt.axvline(x=0, color='black', linewidth=0.8)
20 plt.grid(axis='x', linestyle='--', alpha=0.5)
21 plt.show()
```

=== ПРИБЫЛЬ ПО ПОДКАТЕГОРИЯМ (от худших к лучшим) ===

		Total_Sales	Total_Profit	Avg_Discount	Order_Count
Category	Sub-Category				
Furniture	Tables	206965.53	-17725.48	0.26	319
	Bookcases	114880.00	-3472.56	0.21	228
Office Supplies	Supplies	46673.54	-1189.10	0.08	190
	Fasteners	3024.28	949.52	0.08	217
Technology	Machines	189238.63	3384.76	0.31	115
Office Supplies	Labels	12486.31	5546.25	0.07	364
	Art	27118.79	6527.79	0.07	796
	Envelopes	16476.40	6964.18	0.08	254
Furniture	Furnishings	91705.16	13059.14	0.14	957
Office Supplies	Appliances	107532.16	18138.01	0.17	466
	Storage	223843.61	21278.83	0.07	846
Furniture	Chairs	328449.10	26590.17	0.17	617
Office Supplies	Binders	203412.73	30221.76	0.37	1523
	Paper	78479.21	34053.57	0.07	1370
Technology	Accessories	167380.32	41936.64	0.08	775
	Phones	330007.05	44515.73	0.15	889
	Copiers	149528.03	55617.82	0.16	68



Далее: [New interactive sheet](#)

```
1 # 3.1. Сводная таблица по сегментам
2 if 'Segment' in df.columns:
3     segment_stats = df.groupby('Segment').agg(
4         Total_Sales=('Sales', 'sum'),
5         Total_Profit=('Profit', 'sum'),
```

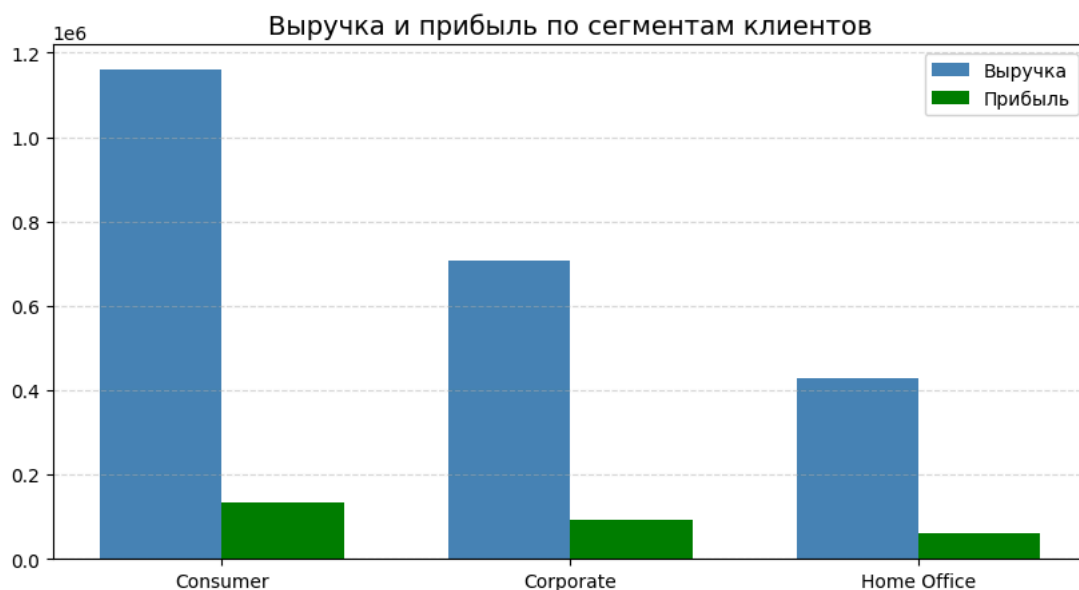
```

6     Avg_Order_Value=('Sales', 'mean'),
7     Order_Count=('Order ID', 'count')
8 ).round(2).sort_values(by='Total_Profit', ascending=False)
9
10    # Добавляем расчётную рентабельность
11    segment_stats['Profit_Margin_%'] = (segment_stats['Total_Profit'] / segment_stats['Total_Sales'] * 100).round(2)
12
13    print("=== АНАЛИЗ ПО СЕГМЕНТАМ КЛИЕНТОВ ===")
14    display(segment_stats)
15
16    # 3.2. Группированная столбчатая диаграмма
17    fig, ax1 = plt.subplots(figsize=(10, 5))
18    x = range(len(segment_stats))
19    width = 0.35
20
21    bars1 = ax1.bar([i - width/2 for i in x], segment_stats['Total_Sales'], width, label='Выручка', color='steelblue',
22    bars2 = ax1.bar([i + width/2 for i in x], segment_stats['Total_Profit'], width, label='Прибыль', color='green')
23
24    ax1.set_title('Выручка и прибыль по сегментам клиентов', fontsize=14)
25    ax1.set_xticks(x)
26    ax1.set_xticklabels(segment_stats.index)
27    ax1.legend()
28    ax1.grid(axis='y', linestyle='--', alpha=0.5)
29    plt.show()
30 else:
31     print("Столбец 'Segment' не найден в датасете.")

```

=== АНАЛИЗ ПО СЕГМЕНТАМ КЛИЕНТОВ ===

	Total_Sales	Total_Profit	Avg_Order_Value	Order_Count	Profit_Margin_%
Segment					
Consumer	1161401.34	134119.21	223.73	5191	11.55
Corporate	706146.37	91979.13	233.82	3020	13.03
Home Office	429653.15	60298.68	240.97	1783	14.03



Далее: [New interactive sheet](#)

Направление Ключевой факт Уточнение к выводу Шага 13 Скидка vs Прибыль $r \approx -0.22$ (умеренная отрицательная связь) Скидки — не единственный фактор убытков; важна категория товара Подкатегории Tables и Bookcases — основные источники убытков Не вся мебель убыточна; проблема точечная Сегменты Consumer = объём, Corporate = маржа Рекомендации по удержанию должны быть сегмент-специфичными

8. Экспорт обработанных данных и отчетов

Для выполнения требований технического задания сформированные данные и визуализации готовы к экспорту:

1. **Обработанный датасет** сохраняется в файл `superstore_processed.csv` (см. ячейку ниже).
2. **Визуализации** можно сохранить как PDF через меню Jupyter/Colab: `File` -> `Print` -> `Save as PDF` (или `Download as PDF`), что закроет требование по PDF-версии дашборда.